

Faculty of Sciences Département of Computer Science



MASTER THESIS

Nauty (trouver un meilleur titre)

Auteur : Rodolphe Prévot

Promoteur : Robin Petit

Directeur : Gwenaël Joret

Academic year 2025–2026

Contents

1	Introduction	2
2	Preliminaries	3
2.1	Algebra Definitions	3
2.2	Graph Definitions	4
2.2.1	Definitions specifics to isomorphisms	5
3	Algorithme Nauty	6
3.1	Forme canonique d'un graphe	6
3.1.1	Fonction de raffinement	7
3.1.2	Target Cell Selection	8
3.1.3	Search Tree construction	9
3.1.4	Node Invariant	9
3.1.5	Implementation strategies	10

Chapter 1

Introduction

Chapter 2

Preliminaries

This chapter introduces all the definitions necessary to understand this document. For further details on the following concepts, refer to Reinhard Diestel's book on graph theory [1].

2.1 Algebra Definitions

Definition 1 (informal). An *isomorphism* is a one-to-one correspondence (mapping) between two sets that preserves binary relationships between elements of the sets.

An *automorphism* is an isomorphism from a set to itself.

Definition 2. A *group* is a set with a binary operation that satisfies the following constraints: the operation is associative, it has an identity element, and every element of the set has an inverse. For example, $(\mathbb{R}, +)$ is a group: it consists of the set of real numbers with addition as the binary operation. It has as identity element : 0, every element has an inverse element (each number with its opposite) and it's associative : for all numbers a, b, c in $\mathbb{R}$: $a + (b + c) = (a + b) + c$.

A *homomorphism* is a transformation of one set into another that preserves in the second set the relations between elements of the first. For example, given two groups, $(G, \cdot)$ and $(H, \circ)$, a *group homomorphism* from $(G, \cdot)$ to $(H, \circ)$ is a function $\phi : G \rightarrow H$ such that $\phi(g_1 \cdot g_2) = \phi(g_1) \circ \phi(g_2)$ for every $g_1, g_2 \in G$.

Let $(G, *)$ be a group with identity element e , and let X be a non-empty set. A *group action* of G on X exists if there is a map

$$\cdot : G \times X \rightarrow X, \quad (g, x) \mapsto g \cdot x$$

such that the following two properties hold:

1. Identity: $\forall x \in X, \quad e \cdot x = x,$
2. Compatibility: $\forall g, h \in G, \forall x \in X, \quad (g * h) \cdot x = g \cdot (h \cdot x).$

As an example, let S_n denote the *symmetric group* acting on a set X . We indicate the action of elements of S_n by exponentiation: for $x \in X$ and $g \in S_n$, x^g denotes the image of x under g . The same notation is used to indicate the induced action on complex structures derived from X . As additional notation, for $W \subseteq V$, we write $W^g = \{w^g : w \in W\}$.

Definition 3. An *equivalence relation*, denoted by $\sim$, on a set X is a binary relation that is reflexive, symmetric, and transitive.

For any element $x \in X$, the *equivalence class* of x under $\sim$ is the subset

$$[x] = \{y \in X \mid y \sim x\}.$$

The *quotient set* of X by $\sim$, denoted by

$$X/\sim = \{[x] \mid x \in X\},$$

is the set of all equivalence classes of X under $\sim$.

2.2 Graph Definitions

Definition 4. A *graph* is a pair $G = (V, E)$ of sets satisfying $E \subseteq \binom{V}{2}$; thus, the elements of E are 2-element subsets of V . The elements of V are the *vertices* of the graph G , the elements of E are its *edges*.

Let V^* denote the set of finite sequences of vertices. For $\sigma \in V^*$, $|\sigma|$ denotes the number of components of σ . If $\sigma = (v_1, v_2, \dots, v_k) \in V^*$ and $w \in V$ then $\sigma \parallel w$ denotes $(v_1, v_2, \dots, v_k, w)$. Furthermore, for $0 \leq s \leq k$, $[\sigma]_s := (v_1, v_2, \dots, v_s)$. The ordering $\leq$ on finite sequences is the lexicographic order: If $\sigma = (v_1, v_2, \dots, v_k)$ and $\sigma' = (v'_1, v'_2, \dots, v'_l)$, then $\sigma \leq \sigma'$ if σ is a prefix of σ' or there is some $j \leq \min\{k, l\}$ such that $v_i = v'_i$ for $i < j$ and $v_j < v'_j$.

An acyclic graph (one not containing any cycles) is called a *forest*. A connected forest is called a *tree*.

Let $G = (V, E)$ be a graph. Let $\sim$ be an equivalence relation on V . The *quotient graph* of G with respect to $\sim$, denoted $G/\sim$, is a graph (V', E') such that :

1. $V' = V/\sim = \{[v] \mid v \in V\}$
2. $E' = \{[u][v] \mid uv \in E\}$

Definition 5. We let $\mathcal{G}_n$ be the set of all graphs on n vertices (without loss of generality, say $V = \{1, 2, \dots, n\}$). $\mathcal{G}$ represents the set of all graphs.

2.2.1 Definitions specifics to isomorphisms

Definition 6. A *colouring* of V (or of $G \in \mathcal{G}$) is a surjective function π from V onto $\{1, 2, \dots, k\}$ for some k . The number of colours, i.e. k , is denoted by $|\pi|$. A cell of π is the set of vertices with some given colour; that is, $\pi^{-1}(j)$ for some j with $1 \leq j \leq |\pi|$. We let Π be the set of all colourings.

If π, π' are colourings, then π' is *finer than or equal* to π , written $\pi' \preceq \pi$, if $\pi(v) < \pi(w) \Rightarrow \pi'(v) < \pi'(w)$ for all $v, w \in V$ (This implies that each cell of π' is a subset of a cell of π).

Here, the colouring is not related to a *proper colouring*, π is related to a *partitioning*.

A *discrete colouring* is a colouring in which each cell is a singleton, in which case $|\pi| = n$.

A *coloured graph* is a pair (G, π) where π is a colouring of G .

A colouring of a graph is called *equitable* if any two vertices of the same colour are adjacent to the same number of vertices of each colour. This is also called an *equitable/stable partition*.

Definition 7. Let S_n denote the *symmetric group* acting on V .

There are several notations to consider:

1. if π is a colouring of V , then π^g is the colouring with $\pi^g(v^g) = \pi(v)$ for each $v \in V$.
2. if (G, π) is a coloured graph, then $(G, \pi)^g = (G^g, \pi^g)$.

These properties show that two coloured graphs (G, π) and (G', π') are isomorphic iff there exists $g \in S_n$ such that $(G', \pi') = (G, \pi)^g$. In this case, we write $(G, \pi) \cong (G', \pi')$.

$\text{Aut}(G, \pi) = \{g \in S_n : (G, \pi)^g = (G, \pi)\}$; $\text{Aut}(G, \pi)$ is the *Automorphism group* of coloured graph (G, π) .

Definition 8. The *canonical form* is a function

$$C : \mathcal{G} \times \Pi \rightarrow \mathcal{G} \times \Pi$$

such that for all $G \in \mathcal{G}, \pi \in \Pi$ and $g \in S_n$:

1. $C(G, \pi) \cong (G, \pi)$.
2. $C(G^g, \pi^g) = C(G, \pi)$.

Chapter 3

Algorithme Nauty

[**Rodolphe:** je devrais peut-être mettre une vraie explication pour graph6 quelque part non?] [**Rodolphe:** note pour moi-même: Je dois caser à un endroit le fait que si on prend n'importe quel feuille du graphe originel et que j'exécute l'algorithme dessus ça va me donner exactement la même chose: preuve]

Dans cette partie, nous allons nous intéresser fortement à l'algorithme général de McKay: Nauty.

Dans un premier temps, nous allons nous attarder sur la création de la forme canonique d'un graphe de cette algorithme de manière théorique. Ensuite, nous allons expliciter l'algorithme en lui-même, et enfin nous allons présenter une implémentation de celui-ci.

3.1 Forme canonique d'un graphe

Tout d'abord, l'algorithme en lui-même est créé sur base d'un arbre très spécifique: un search Tree [**Rodolphe:** déf de search tree au dessus + image]. Cependant, sa particularité est que les feuilles de cet arbre va correspondre à une séquence de vertex d'un coloured graph rodolphe déf de coloured graph au dessus. And as we move down the tree, the sequences grow longer. These sequences define a colouring of the graph obtained by first assigning unique colours to the vertices in the sequence and then deducing, in a "controlled" manner, the colouring of the remaining vertices. The leaves of this tree correspond to sequences whose derived colouring is discrete. [**Rodolphe:** def de discrète au dessus] La racine de cet arbre sera donc la séquence "vide" ; une séquence où il n'y aura aucun vertex avec une unique couleur (créé par notre algorithme). [**Rodolphe:** Je dois retravailler cette phrase parce qu'elle n'est pas très clair]

Pour construire correctement cet arbre et trouver notre forme canonique, nous aurons besoin de quelques fonctions :

- *Une fonction de raffinement: À partir d'une séquence de vertex(partition), cette fonction va nous permettre de construire une nouvelle séquence de vertex(partition) plus fine que la précédente.*
- *Une fonction de sélection de cellule(target cell Selector): À partir d'une séquence de vertex(partition), cette fonction va nous permettre de choisir une cellule de cette partition pour créer "l'enfant" de la partition précédente*
- *Une fonction de node invariant: Une fonction qui va permettre de comparer de manière efficace les différentes feuilles ou Nodes de notre search tree pour trouver la forme canonique du graphe.*

Ces 3 fonctions seront les fonctions de base pour construire notre search tree. Cependant, il en existe d'autres qui sont plus spécifiques à l'implémentation de l'algorithme et qui seront expliquées dans la partie implémentation de ce document. Avant donc de parler de l'implémentation de l'algorithme, nous allons expliquer plus en détail ces fonctions de base et comment elles sont utilisées pour construire notre search tree.

[Rodolphe: Je mettrais bien une sorte d'image pour expliquer comment on construit le search tree à partir de ces fonctions de base, en mode "on part de la racine, on applique la fonction de sélection de cellule pour choisir une cellule, ensuite on applique la fonction de raffinement pour raffiner la partition, ensuite on applique la fonction de node invariant pour calculer l'invariant du noeud, et ensuite on répète ce processus pour les enfants du noeud"]

[Rodolphe: Pour toutes les définitions, j'ai fait : Définition formel du papier puis définition informel/ ce que ça signifie vraiment dis moi t'en penses quoi, sinon je laisse juste formel alors]

3.1.1 Fonction de raffinement

Definition 9. The *Refinement function* is a function

$$R : \mathcal{G} \times \Pi \times V^* \rightarrow \Pi$$

such that, for all $G \in \mathcal{G}$, $\pi \in \Pi$ and $\sigma \in V^*$,

1. $R(G, \pi, \sigma) \preceq \pi$.
2. if $v \in \sigma$, then $\{v\}$ is a cell of $R(G, \pi, \sigma)$.
3. for any $g \in S_n$, we have $R(G^g, \pi^g, \sigma^g) = R(G, \pi, \sigma)^g$.

Ce qui signifie:

1. la fonction de raffinement renvoie un coloring plus fin que π .
2. si un vertex(v) est dans la séquence de vertex(σ), alors il doit être dans une cellule de taille 1 dans le coloring raffiné.
3. Si nous prenons une permutation de notre graphe, notre colouring, et notre séquence de vertex et que nous appliquons la fonction de raffinement à ces éléments, alors nous allons obtenir le même résultat que si nous appliquons la fonction de raffinement à notre graphe, notre coloring, et notre séquence de vertex, et que nous prenons ensuite la même permutation du résultat.

[Rodolphe: Je vais surement parler un peu plus en profondeur ici, et peut-être déjà expliqué les stratégies d'implémentation dans cette partie]

3.1.2 Target Cell Selection

Definition 10. The *Target cell selector function* is a function

$$T : \mathcal{G} \times \Pi \times V^* \rightarrow 2^V$$

such that, for all $G \in \mathcal{G}, \pi \in \Pi$ and $\sigma \in V^*$

1. if $R(G, \pi_0, \sigma)$ is discrete, then $T(G, \pi_0, \sigma) = \emptyset$.
2. if $R(G, \pi_0, \sigma)$ is not discrete, then $T(G, \pi_0, \sigma)$ is a non-singleton cell of $R(G, \pi_0, \sigma)$.
3. for any $g \in S_n$, we have $T(G^g, \pi^g, \sigma^g) = T(G, \pi, \sigma)^g$.

Ce qui signifie:

1. Si la fonction de raffinement renvoie une partition discrète(une feuille de notre search tree), alors *Target cell selector function* ne doit pas renvoyer de cellule. → vide.
2. Si la fonction de raffinement renvoie une partition non-discrète(un node de notre search tree), alors *Target cell selector function* renvoie une cellule de cette partition qui n'a pas qu'un seul élément.
3. Si nous prenons une permutation de notre graphe, notre colouring, et notre séquence de vertex et que nous appliquons *Target cell selector function* à ces éléments, alors nous allons obtenir le même résultat que si nous appliquons *Target cell selector function* à notre graphe, notre coloring, et notre séquence de vertex, et que nous prenons ensuite la même permutation du résultat.

[Rodolphe: Je vais surement parler un peu plus en profondeur ici, et peut-être déjà expliqué les stratégies d'implémentation dans cette partie]

3.1.3 Search Tree construction

Now that all the formal definitions of the tree have been established, we can define the search tree $\mathcal{T}(G, \pi_0)$, which depends on an initial coloured graph (G, π_0) . All the nodes of the tree are elements of V^* . This search tree has some characteristics to keep in mind:

- The root of $\mathcal{T}(G, \pi_0)$ is the empty sequence $()$.
- if σ is a node of $\mathcal{T}(G, \pi_0)$, let $W = T(G, \pi_0, \sigma)$. Then the children of σ are $\{\sigma \parallel w : w \in W\}$.
- a node σ of $\mathcal{T}(G, \pi_0)$ is a leaf iff $R(G, \pi_0, \sigma)$ is discrete.
- for any node σ of $\mathcal{T}(G, \pi_0)$, define $\mathcal{T}(G, \pi_0, \sigma)$ to be the subtree of $\mathcal{T}(G, \pi_0)$ consisting of σ and all its descendants.

Pour montrer le bon fonctionnement de l'algorithme, du pruning que nous allons mettre en place et pour prouver que nous allons trouver la forme canonique du graphe, nous allons devoir prouver quelques propriétés de ce search tree.

Lemma 1. *For any $G \in \mathcal{G}$, $\pi_0 \in \Pi$, $g \in S_n$ we have $\mathcal{T}(G^g, \pi_0^g) = \mathcal{T}(G, \pi_0)^g$.*

[Rodolphe: Faire la preuve]

Corollary 2. *Let σ be a node of $\mathcal{T}(G, \pi_0)$ and let $g \in \text{Aut}(G, \pi_0)$. Then σ^g , is a node of $\mathcal{T}(G, \pi_0)$ and $\mathcal{T}(G, \pi_0, \sigma^g) = \mathcal{T}(G, \pi_0, \sigma)^g$.*

[Rodolphe: Faire la preuve]

Lemma 3. *Let σ be a node of $(\mathcal{G}, \pi_0)$ and let $\pi = R(G, \pi_0, \sigma)$. Then $\text{Aut}(G, \pi)$ is the point-wise stabilizer of σ in $\text{Aut}(G, \pi_0)$.*

[Rodolphe: Faire la preuve]

Maintenant que nous avons bien défini notre search tree, nous allons pouvoir parler de la fonction de node invariant et de comment elle va nous permettre de trouver la forme canonique du graphe.

3.1.4 Node Invariant

Definition 11. Let Ω be some totally ordered set. A *Node invariant* is a function

$$\phi : \mathcal{G} \times \Pi \times V^* \rightarrow \Omega$$

Such that, for any $\pi_0 \in \Pi$, $G \in \mathcal{G}$ and distinct $\sigma, \sigma' \in \mathcal{T}(G, \pi_0)$.

The function ϕ has a few properties to take into account:

1. if $|\sigma| = |\sigma'|$ and $\phi(G, \pi_0, \sigma) < \phi(G, \pi_0, \sigma')$, then for every leaf $\sigma_1 \in \mathcal{T}(G, \pi_0, \sigma)$ and leaf $\sigma'_1 \in \mathcal{T}(G, \pi_0, \sigma')$ we have $\phi(G, \pi_0, \sigma_1) < \phi(G, \pi_0, \sigma'_1)$.

2. if $\pi = R(G, \pi_0, \sigma)$ and $\pi' = R(G, \pi_0, \sigma')$ are discrete,
then $\phi(G, \pi_0, \sigma) = \phi(G, \pi_0, \sigma') \Leftrightarrow G^\pi = G^{\pi'}$ (equality relation).
3. for any $g \in S_n$, we have $\phi(G^g, \pi_0^g, \sigma^g) = \phi(G, \pi_0, \sigma)$.
4. Leaves σ, σ' are *equivalent* if $\phi(G, \pi_0, \sigma) = \phi(G, \pi_0, \sigma')$. This is the case, if there is a unique $g \in \text{Aut}(G, \pi_0)$ such that $\sigma^g = \sigma'$, with $g = R(G, \pi_0, \sigma')R(G, \pi_0, \sigma)^{-1}$. **[Rodolphe: Je pense que je vais retirer ça parce que je préfère parler de tout ce qui est groupe d'automorphisme dans la section prévue pour, t'en penses quoi ?]**

Ce qui signifie:

1. Si deux séquences de vertex sont égaux ($|\sigma| = |\sigma'|$) et que $\phi(G, \pi_0, \sigma) < \phi(G, \pi_0, \sigma')$, Alors cette inégalité se propage à toutes les feuilles descendantes. Aucune feuille descendante de σ ne peut être plus grande que n'importe quelle feuille descendante de σ' .
2. Si nous avons 2 colourings égaux alors, leur invariant sont égaux si et seulement si ils produisent exactement le même canonical labelling ; Leur appliquer la permutation associé à leur colouring produira exactement le même graphe.
3. Si nous prenons une permutation de notre graphe, notre colouring, et notre séquence de vertex et que nous appliquons *Node invariant* à ces éléments, c'est équivalent à appliquer *Node invariant* à notre graphe, notre coloring, et notre séquence de vertex, de base.

We can define the canonical form:

$$\phi^*(G, \pi_0) = \max \{ \phi(G, \pi_0, \sigma) : \sigma \text{ is a leaf of } \mathcal{T}(G, \pi_0) \},$$

3.1.5 Implementation strategies

[Rodolphe: Par forcément sur de le mettre là though, donc soit là et ça fait vraiment histoire dans le sens Théorie into pratique ou bien melting pot et fusionne les 2 parties]

Maintenant que nous avons toute la théorie en main, nous pouvons commencer à parler de l'implémentation de l'algorithme. Notre implémentation à nous se base sur la les définitions de l'algorithme mais sans être une pâle copie de son code source.

Implementation Refinement function

[Rodolphe: Globalement ici, les 2 fonctions c'est vraiment du copier coller de mckay, je peux laisser comme ça tu crois ?]

In order to ensure that the properties of the **refinement function** defined in the search tree are respected, we must implement an algorithm that outputs a colouring that is equal to (or finer) than the original colouring. To this end, we rely on a theorem: for every colouring π , there exists a unique (up to cell ordering) coarsest equitable colouring π' such that $\pi' \preceq \pi$.

Pour créer notre fonction de raffinement, nous aurons besoin de 2 fonctions : Une fonction qui va raffiner un colouring pour que ça devienne un colouring équitable, et une fonction d'individualisation.

Definition 12. The function $F(G, \pi, \alpha)$ is our refinement algorithm (implemented in a label-invariant manner).

pseudo-code of $F(G, \pi, \alpha)$ (from [4])

```

1: Data:  $\pi$  is the input colouring and  $\alpha$  is a sequence of some cells of  $\pi$ 
2: Result: the final value of  $\pi$  is the output colouring(equitable)
3: while  $\alpha$  is not empty and  $\pi$  is not discrete do
4:   Remove some element  $W$  from  $\alpha$ 
5:   for each cell  $X$  of  $\pi$  do
6:     Let  $X_1, \dots, X_k$  be the fragments of  $X$  distinguished according to
       the number of edges from each vertex to  $W$ 
7:     Replace  $X$  by  $X_1, \dots, X_k$  in  $\pi$ 
8:     if  $X \in \alpha$  then
9:       Replace  $X$  by  $X_1, \dots, X_k$  in  $\alpha$ 
10:    else
11:      Add all but one of the largest of  $X_1, \dots, X_k$  to  $\alpha$ 
12:    end if
13:  end for
14: end while

```

[Rodolphe: Faire une explication plus détaillée de l'algorithme ?]

Definition 13. The *individualized function* $I : \Pi \times V \rightarrow \Pi$

such that, if v is a vertex in a non-singleton cell of π and $\pi' = I(\pi, v)$, then for $w \in V$,

$$\pi'(w) = \begin{cases} \pi(w), & \text{if } \pi(w) < \pi(v) \text{ or } w = v; \\ \pi(w) + 1, & \text{otherwise.} \end{cases}$$

We say that the vertex v is individualized in π . We see that $I(\pi, v)$ differs from π in that a unique colour has been given to vertex v .

[Rodolphe: Même chose ici ? Faire une explication plus détaillée de la fonction ?]

Now we can define a refinement function. For a sequence of vertices $v_1, v_2, \dots$, define

- $R(G, \pi_0, ()) = F(G, \pi_0, \text{a list of all the cells of } \pi_0)$
- $R(G, \pi_0, (v_1)) = F(G, I((R(G, \pi_0, ())), v_1), \{v_1\})$
- $R(G, \pi_0, (v_1, v_2)) = F(G, I((R(G, \pi_0, (v_1))), v_2), \{v_2\})$
- $R(G, \pi_0, (v_1, v_2, v_3)) = F(G, I((R(G, \pi_0, (v_1, v_2))), v_3), \{v_3\})$
- and so on.

The algorithm satisfies the required properties of a refinement function and performs well in practice. However, the refinement step is the most computationally expensive part of the algorithm, which is why special care must be taken in its implementation. C'est pourquoi, pour réduire les temps de calculs, nous allons utiliser des méthodes de pruning dont nous parlerons un peu plus tard.

Implementation Target Cell Selector

[**Rodolphe:** Même chose ici, c'est un peu du copier coller du papier de mckay, je laisse ça comme ça, ou alors je rajoute un peu de sel avec ?]

For the implementation of the **target cell selection function** in the construction of the search tree, several strategies can be adopted. Choosing the right target cell at each step is important because it influences the shape and depth of the tree, and therefore the overall performance of the search.

In the original version of *Nauty*, McKay recommended selecting the first smallest non-singleton cell, as it increases the chance that the selected cell is part of an orbit of the group that stabilizes the current sequence [3]. However, later studies showed that in certain cases, it is better to simply select the first non-singleton cell, regardless of its size [2].

As a result, *Nauty* now supports two strategies for target cell selection that we can choose:

1. Always select the first smallest non-singleton cell.
2. Select the first cell that is joined in a non-trivial way to the largest number of other cells, where a non-trivial join means the number of edges between two cells is strictly between 0 and the maximum possible.

Dans notre implémentation, nous n'utiliserons que la première stratégie.

[**Rodolphe:** S'il reste du temps, intéressant de faire une comparaison entre les 2 stratégies ?]

Implementation Node Invariant

Now that the search tree is implemented, we need to define the **node invariant function**, which will allow us to determine the canonical form. To compute this function, we can rely on two related sources:

1. For each node σ , there is a colouring $R(G, \pi_0, \sigma)$. We can use its properties to extract the number and size of the cells, as well as combinatorial properties of the coloured graph.
2. We can use the intermediate states of the colouring computation from the parent node (in the refinement algorithm), such as the position and size of the cells produced during the refinement procedure, and various edge counts determined during the computation. If $f(\sigma)$ is a function of this information—computed during the calculation of $R(G, \pi_0, \sigma)$ and from the resulting coloured graph—then the vector $(f([\sigma]_0), f([\sigma]_1), \dots, f(\sigma))$, with lexicographic ordering, satisfies the characteristics required for a node invariant. In *Nauty*, the second source is used: the value of $f(\sigma)$ is an integer, and the pruning rules are applied as each node is computed.

Another potential method is based on the concept of the *quotient graph* $Q(G, \pi)$ (see [Definition 4](#)), assuming that the colouring π of graph G is equitable. All vertices of $Q(G, \pi)$ are cells of π , and they include information about the size and number of the cell. For any two cells $C_1, C_2 \in \pi$, possibly equal, the corresponding vertices in $Q(G, \pi)$ are connected by an edge labelled with the number of edges in G between C_1 and C_2 . The node invariant already computed by *Nauty* is a deterministic function of the sequence of quotient graphs $Q(G, R(G, \pi_0, [v]_i))$ for $i = 0, \dots, |v|$. Therefore, it would be possible to use the sequence of quotient graphs to obtain information about the cells, but this would be far too costly in both time and memory.

Dans notre implémentation, nous allons utiliser la première source décrite au-dessus. Nous utiliserons le nombre et la taille des cellules et appliquerons des propriétés combinatoires du graphe coloré pour calculer notre invariant de noeud. Plusieurs méthodes d'invariants ont été mises en place afin de tester leur efficacité et leur rapidité de calcul. Nous allons présenter les différentes méthodes que nous avons implémentées et les résultats que nous avons obtenus pour chacune d'entre elles dans la section des résultats.

[**Rodolphe:** Bon je savais pas trop quoi dire sur la fin, dans le sens où je sais pas si je présente déjà ici mes méthodes d'invariants, (j'ai ici que le nombre de triangle dans les cellules mais je rajouterai les prochaines que vais implémenter.), ou alors comme j'ai dit, je mets dans plus dans les résultats mais en faisant ce commentaire j'ai l'impression que c'est quand même mieux ici]

Implementation pruning

To avoid generating unnecessary information in our search tree, which would result in an overly large tree, we aim to construct the most optimized tree possible. To achieve this, we can apply *pruning* techniques to the tree: eliminating redundant branches so that only a fraction of the original tree needs to be generated. We can define three types of pruning strategies to optimize our construction:

1. Suppose σ, σ' are distinct nodes of $T(G, \pi_0)$ with $|\sigma| = |\sigma'|$ and $\phi(G, \pi_0, \sigma) > \phi(G, \pi_0, \sigma')$. Operation $P_A(\sigma, \sigma')$ is to remove $T(G, \pi_0, \sigma')$.
2. Suppose σ, σ' are distinct nodes of $T(G, \pi_0)$ with $|\sigma| = |\sigma'|$ and $\phi(G, \pi_0, \sigma) \neq \phi(G, \pi_0, \sigma')$. Operation $P_B(\sigma, \sigma')$ is to remove $T(G, \pi_0, \sigma')$.

[Rodolphe: J'ai retiré PC vu que ce n'est pas encore implémenté, mais je le rajouterai quand on parlera de aut(2ème partie donc)]

Despite these pruning strategies, we must ensure that our search tree, along with the node invariant function, remains functionally correct. To this end, we require a theorem that guarantees the validity of the overall construction.

Theorem 4. *Consider any $G \in \mathcal{G}$ and $\pi_0 \in \Pi$*

- *Suppose any sequence of operations of the form $P_A(\sigma, \sigma')$ or $P_C(\sigma, g)$ are performed.
Then there remains at least one leaf σ_1 with $\phi(G, \pi_0, \sigma_1) = \phi^*(G, \pi_0)$.*
- *Let σ_0 be some fixed leaf of $T(G, \pi_0)$. Suppose any sequence of operations of the form $P_B(\sigma, \sigma')$ $P_C(\sigma'', g)$ are performed, where $\phi(G, \pi_0, \sigma'') \neq \phi(G, \pi_0, [\sigma_0]_{|\sigma''|})$. Let $g_1, \dots, g_k$ be the automorphisms used in the operations P_C that were performed, and let $A = \{g \in \text{Aut}(G, \pi_0) : v_0^g \text{ is a remaining leaf}\}$. Then $\text{Aut}(G, \pi_0)$ is generated by $\{g_1, \dots, g_k\} \cup A$.*

[Rodolphe: Faudrait que je refasse la preuve ici par moi-même mais ça parle de Pc, est-ce que je retirais la preuve, je ferai une mini preuve pour expliquer que même avec Pa et PB c'est good, et puis je ferai une mini preuve pour Pc quand on parlera du groupe d'automorphisme ?]

Bibliography

- [1] R. Diestel. *Graph Theory*, volume 173 of *Graduate Texts in Mathematics*. Springer, Heidelberg, 6th edition, 2025. ISBN: 978-3-662-53621-6.
- [2] A. Kirk. Efficiency considerations in the canonical labelling of graphs. Technical report, Technical report TR-CS-85-05, Computer Science Department, Australian, 1985.
- [3] B. D. McKay et al. Practical graph isomorphism, 1981.
- [4] B. D. McKay and A. Piperno. Practical graph isomorphism, II. *Journal of symbolic computation*, 60:94–112, 2014.